ASSIGNMENT

Self Learning Assignment 01

1. Write a C++ program to calculate Simple and Compound interest.

Code:-

```
#include <iostream>
#include <cmath>
using namespace std;

double calculateSimpleInterest(double p, double r, double t) {
    return (p * r * t) / 100.0;
}
double calculateCompoundInterest(double p, double r, double t) {
    double amount = p * pow((1 + r / 100.0), t);
    return amount - p;
}
int main() {
    double principal, rate, time;

    cout << "Enter Principal Amount: ";
    cin >> principal;
    cout << "Enter Interest Rate (%): ";
    cin >> rate;
    cout << "Enter Time (Years): ";
    cin >> time;

    double si = calculateSimpleInterest(principal, rate, time);
    double ci = calculateCompoundInterest(principal, rate, time);
    cout << "\n--- Results ---" << endl;
    cout << "Simple Interest:   " << si << endl;
    cout << "Compound Interest: " << ci << endl;
    return 0;
}
```

O/P:-

```
Enter Principal Amount: 10000
Enter Interest Rate (%): 11.11
Enter Time (Years): 3

--- Results ---
Simple Interest:   3333
Compound Interest: 3717.01
```

2. **Write a C++ Program to transpose one matrix and display both the matrices with proper prompt.**

Code:-

```
#include <iostream>
using namespace std;
int main() {
    // Define a maximum size for the matrices
    int rows, cols;
    int matrix[10][10], transpose[10][10];

    // 1. Input Dimensions
    cout << "Enter rows and columns (max 10): ";
    cin >> rows >> cols;

    // 2. Input Matrix Elements
    cout << "\nEnter elements of the matrix:\n";
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << "Enter element [" << i << "][" << j << "]: ";
            cin >> matrix[i][j];
        }
    }

    // 3. Logic: Transpose the Matrix
    // The element at [i][j] moves to [j][i]
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            transpose[j][i] = matrix[i][j];
        }
    }

    // 4. Display Original Matrix
    cout << "\n--- Original Matrix ---\n";
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << matrix[i][j] << "  ";
        }
        cout << "\n";
    }

    // 5. Display Transposed Matrix
    // Note: The loop now goes up to 'cols' for rows and 'rows' for columns
    std::cout << "\n--- Transposed Matrix ---\n";
    for (int i = 0; i < cols; ++i) {
```

```
        for (int j = 0; j < rows; ++j) {
            cout << transpose[i][j] << "  ";
        }
        cout << "\n";
    }

    return 0;
}
```

O/P:-

```
Enter rows and columns (max 10): 3
3

Enter elements of the matrix:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [0][2]: 3
Enter element [1][0]: 4
Enter element [1][1]: 5
Enter element [1][2]: 6
Enter element [2][0]: 7
Enter element [2][1]: 8
Enter element [2][2]: 9

--- Original Matrix ---
1  2  3
4  5  6
7  8  9

--- Transposed Matrix ---
1  4  7
2  5  8
3  6  9
```

3. **Write a C++ program to process students' marks with the help of classes. The class has private variables for name, mark 1, mark 2, mark 3. It has two member functions - getdata() - to get input and result() - to print the results. All subjects mark must be >= 50 for pass otherwise fail. Implement this program for n number of students.**

Code:-

```
#include <iostream>
using namespace std;

class Student {
private:
    char name[50];
    int mark1, mark2, mark3;
public:
    void getData() {
        cout << "Enter student name: "<<endl;
        cin >> name;
        cout << "Enter Mark 1: "<<endl;
        cin >> mark1;
        cout << "Enter Mark 2: "<<endl;
        cin >> mark2;
        cout << "Enter Mark 3: "<<endl;
        cin >> mark3;
    }
    void result() const {
        cout << "Name: " << name << endl;
        cout << " | Marks: " << mark1 << ", " << mark2 << ", " << mark3 << endl;
        if (mark1 >= 50 && mark2 >= 50 && mark3 >= 50) {
            cout << " | Status: PASS";
        } else {
            cout << " | Status: FAIL";
        }
        cout << endl;
    }
};

int main() {
    int n;
    cout << "Enter the number of students: ";
    cin >> n;
    Student* studentList = new Student[n];
    for (int i = 0; i < n; i++) {
        cout << "\n--- Input for Student " << (i + 1) << " ---";
        studentList[i].getData();
```

```
    }
    cout << "\n--- Student Results ---";
    for (int i = 0; i < n; i++) {
        studentList[i].result();
    }
    delete[] studentList;
    return 0;
}
```

O/P:-

```
Enter the number of students: 3

--- Input for Student 1 ---Enter student name:
sanket
Enter Mark 1:
98
Enter Mark 2:
76
Enter Mark 3:
56

--- Input for Student 2 ---Enter student name:
shubham
Enter Mark 1:
87
Enter Mark 2:
67
Enter Mark 3:
99

--- Input for Student 3 ---Enter student name:
xyz
Enter Mark 1:
56
Enter Mark 2:
45
Enter Mark 3:
67

--- Student Results ---Name: sanket
 | Marks: 98, 76, 56
 | Status: PASS
Name: shubham
 | Marks: 87, 67, 99
 | Status: PASS
Name: xyz
 | Marks: 56, 45, 67
 | Status: FAIL
```

4. **Write a C++ program that allows the user to input 5 grades, i.e. marks between 0-100. The program must calculate the average mark, and state the number of marks less than 65.**

Code:-

```
#include <iostream>
using namespace std;

void inputGrades(int grades[], int size) {
    for (int i = 0; i < size; i++) {
        cout << "Enter marks:  " << i + 1 << " (0-100): ";
        cin >> grades[i];
        while (grades[i] < 0 || grades[i] > 100) {
            cout << "Invalid input. Please enter a mark between 0 and 100: ";
            cin >> grades[i];
        }
    }
}

double calculateAverage(int grades[], int size) {
    double sum = 0;
    for (int i = 0; i < size; i++) {
        sum += grades[i];
    }
    return sum / size;
}

int countFailingMarks(int grades[], int size) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (grades[i] < 65) {
            count++;
        }
    }
    return count;
}

int main() {
    const int SIZE = 5;
    int grades[SIZE];
    cout << "--- Grade Analysis System ---" << endl;

    inputGrades(grades, SIZE);

    double average = calculateAverage(grades, SIZE);
```

```
    int below65 = countFailingMarks(grades, SIZE);

    cout << "\n--- Results ---" << endl;
    cout << "Average Marks: " << average << endl;
    cout << "Number of marks less than 65: " << below65 << endl;
    return 0;
}
```

O/P:-

```
--- Grade Analysis System ---
Enter marks:  1 (0-100): 89
Enter marks:  2 (0-100): 98
Enter marks:  3 (0-100): 76
Enter marks:  4 (0-100): 98
Enter marks:  5 (0-100): 67

--- Results ---
Average Marks: 85.6
Number of marks less than 65: 0
```

5. **Write a menu driven C++ program to add, subtract and multiply given two matrices defined in class. Modify your program using operator overloading method.**

Code:-

```
#include<iostream>
using namespace std;

class Matrix{
   int a[3][3];
public:
   void accept(){
      cout<<"Enter matrix elements:\n";
      for(int i=0;i<3;i++){
         for(int j=0;j<3;j++){
            cin>>a[i][j];
         }
      }
   }
   void display(){
      for(int i=0;i<3;i++){
         for(int j=0;j<3;j++){
            cout<<a[i][j]<<" ";
         }
         cout<<endl;
      }
   }
   // Addition
   Matrix operator + (Matrix m){
      Matrix temp;
      for(int i=0;i<3;i++){
         for(int j=0;j<3;j++){
            temp.a[i][j] = a[i][j] + m.a[i][j];
         }
      }
      return temp;
   }

   // Subtraction
   Matrix operator - (Matrix m){
      Matrix temp;
      for(int i=0;i<3;i++){
         for(int j=0;j<3;j++){
            temp.a[i][j] = a[i][j] - m.a[i][j];
         }
```

```
        }
        return temp;
    }

    // Multiplication
    Matrix operator * (Matrix m){
        Matrix temp;
        for(int i=0;i<3;i++){
            for(int j=0;j<3;j++){
                temp.a[i][j] = 0;
                for(int k=0;k<3;k++){
                    temp.a[i][j] += a[i][k] * m.a[k][j];
                }
            }
        }

        return temp;
    }
};
int main(){
    Matrix m1, m2, result;
    int choice;
    cout<<"Enter Matrix 1\n";
    m1.accept();
    cout<<"Enter Matrix 2\n";
    m2.accept();
    do{
        cout<<"\n1.Add\n2.Subtract\n3.Multiply\n4.Exit\n";
        cout<<"Enter choice: ";
        cin>>choice;
        switch(choice) {
            case 1:
                result = m1 + m2;
                cout<<"Addition:\n";
                result.display();
                break;

            case 2:
                result = m1 - m2;
                cout<<"Subtraction:\n";
                result.display();
                break;

            case 3:
```

```
            result = m1 * m2;
            cout<<"Multiplication:\n";
            result.display();
            break;
        }
    }while(choice != 4);
    return 0;
}
```

O/P:-

```
Enter Matrix 1
Enter matrix elements:
1
2
3
4
5
6
7
8
9
Enter Matrix 2
Enter matrix elements:
1
2
3
4
5
6
7
8
9

1.Add
2.Subtract
3.Multiply
4.Exit
Enter choice: 1
Addition:
2 4 6
8 10 12
14 16 18

1.Add
2.Subtract
3.Multiply
4.Exit
```

```
1.Add
2.Subtract
3.Multiply
4.Exit
Enter choice: 2
Subtraction:
0 0 0
0 0 0
0 0 0

1.Add
2.Subtract
3.Multiply
4.Exit
Enter choice: 3
Multiplication:
30 36 42
66 81 96
102 126 150

1.Add
2.Subtract
3.Multiply
4.Exit
Enter choice: 4
```

6. **Create a class DONOR that contains donor number, donor name, age address, sex and blood group. Write a Menu driven C++ program to display the number, name and address of the donors for the following categories:**
   a. blood donors having the blood group 0+
   b. blood donors in the age group between 18 to 25
   c. female donors having blood group A in the age between 21 and 24.

Code:-

```
#include<iostream>
#include<cstring>
using namespace std;

class Donor{
   private:
      int dNumber;
      char* dName;
      int dAge;
      char* dAddress;
      char* sex;
      char* dBloodgroup;
   public:
      Donor(){
         dNumber = 0;
         dName = NULL;
         dAge = 0;
         dAddress = NULL;
         sex = NULL;
         dBloodgroup = NULL;
      }
      Donor(int dNum,const char* dNm, int dA,const char* dAdre,const char*
dS,const char* dBG){
         //size allocation
         dName = new char[strlen(dNm)+1];
         dAddress = new char[strlen(dAdre)+1];
         sex = new char[strlen(dS)+1];
         dBloodgroup = new char[strlen(dBG)+1];
         //values are stored
         dNumber = dNum;
         strcpy(dName,dNm);
         dAge = dA;
         strcpy(dAddress,dAdre);
         strcpy(sex,dS);
         strcpy(dBloodgroup,dBG);
```

```
}
//accepting donar data
void acceptData(Donor** dnr, int size){
    int num;
    char nm[30];
    int a;
    char adr[50];
    char sx[10];
    char bloodgrp[5];
    for(int i=0; i<size; i++){
        cout<<"Enter Donar ID: "<<endl;
        cin>>num;
        cout<<"Enter Donar Name: "<<endl;
        cin>>nm;
        cout<<"Enter Donar Address: "<<endl;
        cin>>adr;
        cout<<"Enter Donar Age: "<<endl;
        cin>>a;
        cout<<"Enter Donar Sex: "<<endl;
        cin>>sx;
        cout<<"Enter Donar Blood Group: "<<endl;
        cin>>bloodgrp;
        dnr[i] = new Donor(num, nm, a, adr, sx, bloodgrp);
    }
}

//displaying category 1 donars: O+ donars
void category1(Donor** cnr, int size){
    for(int i=0; i<size; i++){
        if(strcmp(cnr[i]->dBloodgroup,"O+")==0)
        {
            cout<<"----------------------------"<<endl;
            cout<<"Donor Number: "<<cnr[i]->dNumber<<endl;
            cout<<"Donor Name: "<<cnr[i]->dName<<endl;
            cout<<"Donor Address: "<<cnr[i]->dAddress<<endl;
        }
    }
}
//displaying category 2 donars: Age group 18-25
void category2(Donor** cnr, int size){
    for(int i=0; i<size; i++){
        if((cnr[i]->dAge >= 18 )&& (cnr[i]->dAge <= 25 ))
        {
            cout<<"----------------------------"<<endl;
```

```
                cout<<"Donor Number: "<<cnr[i]->dNumber<<endl;
                cout<<"Donor Name: "<<cnr[i]->dName<<endl;
                cout<<"Donor Address: "<<cnr[i]->dAddress<<endl;
            }
        }
    }
    //displaying category 3 donars: Female Donor with BloodGroup A, and Age
between 21-24
    void category3(Donor** cnr, int size){
        for(int i=0; i<size; i++){
            if((strcmp(cnr[i]->dBloodgroup,"A") == 0) && (cnr[i]->dAge >= 21 ) &&
(cnr[i]->dAge <= 24)  && (strcmp(cnr[i]->sex,"Female" ) == 0))
            {
                cout<<"----------------------------"<<endl;
                cout<<"Donor Number: "<<cnr[i]->dNumber<<endl;
                cout<<"Donor Name: "<<cnr[i]->dName<<endl;
                cout<<"Donor Address: "<<cnr[i]->dAddress<<endl;
            }
        }
    }
    //displaying category 4 donars: O+ donars
    void category4(Donor** cnr, int size){
        for(int i=0; i<size; i++){

                cout<<"----------------------------"<<endl;
                cout<<"Donor Number: "<<cnr[i]->dNumber<<endl;
                cout<<"Donor Name: "<<cnr[i]->dName<<endl;
                cout<<"Donor Address: "<<cnr[i]->dAddress<<endl;
        }
    }
};
int main(){
    int size, choice;
    cout<<"Enter total number of Donors: "<<endl;
    cin>>size;
    Donor** dnr = new Donor*[size];
    Donor d;
    d.acceptData(dnr,size);

    do{
        cout<<"1. O+ Donors\n2. Age 18-25 Donors\n3. Female, Blood Group A, Age
21-24 Donors\n4. Display All Donors\n5. Exit"<<endl;
        cin>>choice;
        switch(choice){
```

```
        case 1: d.category1(dnr, size);
                cout<<"----------------------------"<<endl;
                break;

        case 2: d.category2(dnr, size);
                cout<<"----------------------------"<<endl;
                break;

        case 3: d.category3(dnr, size);
                cout<<"----------------------------"<<endl;
                break;

        case 4: d.category4(dnr, size);
                cout<<"----------------------------"<<endl;
                break;

        case 5: //exit
                break;
        default: cout<<"Invalid Input!!!"<<endl;
                break;
      }
   }while(choice!=5);

   return 0;
}
```

O/P:-

Enter total number of Donors:
4
Enter Donar ID:
11
Enter Donar Name:
Sanket
Enter Donar Address:
Baner
Enter Donar Age:
22
Enter Donar Sex:
Male
Enter Donar Blood Group:
O+
Enter Donar ID:
22

Enter Donar Name:
Shubham
Enter Donar Address:
Panchavati
Enter Donar Age:
24
Enter Donar Sex:
Male
Enter Donar Blood Group:
A
Enter Donar ID:
33
Enter Donar Name:
Tej
Enter Donar Address:
Pune
Enter Donar Age:
24
Enter Donar Sex:
Female
Enter Donar Blood Group:
A
Enter Donar ID:
44
Enter Donar Name:
Poonam
Enter Donar Address:
Mumbai
Enter Donar Age:
21
Enter Donar Sex:
Female
Enter Donar Blood Group:
A
1. O+ Donors
2. Age 18-25 Donors
3. Female, Blood Group A, Age 21-24 Donors
4. Display All Donors
5. Exit
1

Donor Number: 11
Donor Name: Sanket
Donor Address: Baner

```
----------------------------
1. O+ Donors
2. Age 18-25 Donors
3. Female, Blood Group A, Age 21-24 Donors
4. Display All Donors
5. Exit
2
----------------------------
Donor Number: 11
Donor Name: Sanket
Donor Address: Baner
----------------------------
Donor Number: 22
Donor Name: Shubham
Donor Address: Panchavati
----------------------------
Donor Number: 33
Donor Name: Tej
Donor Address: Pune
----------------------------
Donor Number: 44
Donor Name: Poonam
Donor Address: Mumbai
----------------------------
1. O+ Donors
2. Age 18-25 Donors
3. Female, Blood Group A, Age 21-24 Donors
4. Display All Donors
5. Exit
3
----------------------------
Donor Number: 33
Donor Name: Tej
Donor Address: Pune
----------------------------
Donor Number: 44
Donor Name: Poonam
Donor Address: Mumbai
----------------------------
1. O+ Donors
2. Age 18-25 Donors
3. Female, Blood Group A, Age 21-24 Donors
4. Display All Donors
5. Exit
```

4

Donor Number: 11
Donor Name: Sanket
Donor Address: Baner

Donor Number: 22
Donor Name: Shubham
Donor Address: Panchavati

Donor Number: 33
Donor Name: Tej
Donor Address: Pune

Donor Number: 44
Donor Name: Poonam
Donor Address: Mumbai

----

1. O+ Donors
2. Age 18-25 Donors
3. Female, Blood Group A, Age 21-24 Donors
4. Display All Donors
5. Exit
5

7. **An electricity board charges the following rates to domestic users:**
   For the first 100 units -. 5.5P per unit
   For next 200 units -. 7.25P per unit
   Beyond 300 units -. 8.75P per unit
   All users are charged a minimum of ₹. 60.
   If the total amount is more than 2. 750 then an additional surcharge of 10% is added on the higher slab units.
   Write a C++ program to read the names of users and number of units consumed and print out the charges with names.

Code:-

```
#include<iostream>
#include<cstring>
using namespace std;

class consumers{
   private:
      char* name;
      int unitsConsumed;
      double bill;
   public:
      consumers(){
         name = NULL;
         unitsConsumed = 0;
         bill=60.00;
      }
      consumers(const char* nm, int units){
         name = new char[strlen(nm)+1];
         strcpy(name,nm);
         unitsConsumed = units;
         bill=60.00;
      }
      void acceptUsers(consumers** cns, int size){
         char n[20];
         int unit;
         for(int i=0; i<size; i++){
            cout<<"Enter User's Name: "<<endl;
            cin>>n;
            cout<<"Enter Units consumed: "<<endl;
            cin>>unit;

            cns[i] = new consumers(n, unit);
         }
      }
      void calculateBill(consumers** cns, int size){
```

```
        for(int i=0; i<size; i++){
            int units=cns[i]->unitsConsumed;
            double finalBill=0, surcharge=0;
            // Calculate amount based on slab
            if (units <= 100){
                finalBill = units * 5.5;
            }
            else if (units <= 300){
                finalBill = (100 * 5.5) + (units - 100) * 7.25;
            }
            else{
                finalBill = (100 * 5.5) + (200 * 7.25) + (units - 300) * 8.75;
            }
            // Minimum charge
            if (finalBill < 60){
                finalBill = 60;
            }
            // Surcharge condition
          if(finalBill > 750){
                surcharge = (finalBill - 750) * 0.10;
            }
            cns[i]->bill = finalBill + surcharge;
        }
    }
    void displayUsers(consumers** cns, int size){
        for(int i=0; i<size; i++){
            cout<<"-------------------------------------"<<endl;
            cout<<"Name: "<<cns[i]->name<<endl;
            cout<<"Units: "<<cns[i]->unitsConsumed<<endl;
            cout<<"Bill: "<<cns[i]->bill<<endl;
            cout<<"-------------------------------------"<<endl;
        }
    }
    ~consumers(){
        delete[] name;
    }
};
int main(){
    int s;
    cout<<"Enter size of users array: "<<endl;
    cin>>s;
    consumers** cns = new consumers*[s];
    consumers c;
    c.acceptUsers(cns, s);
```

```
    c.calculateBill(cns,s);
    c.displayUsers(cns, s);
    //deleting memory to prevent memory leak
      for(int i=0;i<s;i++)
        delete cns[i];

    delete[] cns;

    return 0;
}
```

O/P:-

```
Enter size of users array:
3
Enter User's Name:
sanket
Enter Units consumed:
890
Enter User's Name:
tejas
Enter Units consumed:
86
Enter User's Name:
shubham
Enter Units consumed:
430
-------------------------------------
Name: sanket
Units: 890
Bill: 7803.75
-------------------------------------
-------------------------------------
Name: tejas
Units: 86
Bill: 473
-------------------------------------
-------------------------------------
Name: shubham
Units: 430
Bill: 3376.25
-------------------------------------
```

8. **Implement the calculator through a C++ program (use switch). The program should ask the user to input two operands and an operator. Your program should print the result of performing the required operation on the input operands as the following output:**
   Enter operand 1:5
   Enter operand 2: 10
   Enter operation: /
   The result of 5/10 is 0.5

Code:-

```
#include <iostream>
using namespace std;

int main() {
    double op1, op2;
    char operation;

    cout << "Enter operand 1: ";
    cin >> op1;
    cout << "Enter operand 2: ";
    cin >> op2;
    cout << "Enter operation (+, -, *, /): ";
    cin >> operation;

    cout << "\nThe result of " << op1 << operation << op2 << " is ";

    switch (operation) {
        case '+': cout << op1 + op2; break;
        case '-': cout << op1 - op2; break;
        case '*': cout << op1 * op2; break;
        case '/':
            if (op2 != 0) cout << op1 / op2;
            else cout << "Error (Division by zero)";
            break;
        default: cout << "Invalid operator!";
    }
    cout << endl;

    return 0;
}
```

O/P:-

```
Enter operand 1: 5
Enter operand 2: 10
Enter operation (+, -, *, /): +

The result of 5+10 is 15

c:\Users\solan\AppData\Local\Temp\30753dfc-fe
n\AppData\Local\Temp\30753dfc-fea9-462b-9a45-
culator && "c:\Users\solan\AppData\Local\Temp
ulator
Enter operand 1: 0
Enter operand 2: 10
Enter operation (+, -, *, /): /

The result of 0/10 is 0
```

9. **Create Time Class which contains the members as hour, minute and second. Write a C++ program that accepts member data for one Time class object from the keyboard. Then, ask the user to perform one of the following functions to that Time object:**
   AddHours
   AddMinutes
   AddSeconds
   And finally display the newly modified Time information.

Code:-

```
#include<iostream>
#include<cstring>
using namespace std;

class Time{
   private:
      int hh, mm, ss;
   public:
      //Parameterized constructor with default values
      Time(int h=0, int m=0, int s=0){
         hh=h;
         mm=m;
         ss=s;
      }
      void acceptTime(){
         cout<<"Enter Time in HH:MM:SS format: "<<endl;
         cin>>hh>>mm>>ss;
      }
      void displayTime(){
         cout<<"HH: "<<hh<<" MM: "<<mm<<" SS: "<<ss<<endl;
      }
      void addSeconds(int sec){
         this->ss = ss + sec;
         if(ss >= 60){
            mm += ss/60;
            ss%=60;
         }
         //Adjusting minutes
         if(mm>=60){
            hh += mm/60;
            mm%=60;
         }
      }
       void addMinutes(int min){
         this->mm = mm + min;
```

```
        //Adjusting minutes
        if(mm>=60){
            hh += mm/60;
            mm%=60;
        }
    }
    void addHour(int hr){
        this->hh = hh + hr;
    }
};
int main(){
    int choice,h,m,s;
    Time t;
    t.acceptTime();
    do{
         cout<<"1. Add Hours\n2. Add Minutes\n3. Add Seconds\n4. Display
Time\n5.Exit"<<endl;
    cin>>choice;
    switch(choice){
        case 1: cout<<"Enter Hours: "<<endl;
                cin>>h;
                t.addHour(h);
                break;

        case 2: cout<<"Enter Minutes: "<<endl;
                cin>>m;
                t.addMinutes(m);
                break;

        case 3: cout<<"Enter Seconds: "<<endl;
                cin>>s;
                t.addSeconds(s);
                break;

        case 4: cout<<"Updated Time: "<<endl;
                t.displayTime();

        case 5: //exit
                break;
        default: cout<<"Invalid Input!!!"<<endl;
                break;
    }
    }while(choice!=5);
```

```
    return 0;
}
```

O/P:-

```
Enter Time in HH:MM:SS format:
11
22
33
1. Add Hours
2. Add Minutes
3. Add Seconds
4. Display Time
5.Exit
1
Enter Hours:
2
1. Add Hours
2. Add Minutes
3. Add Seconds
4. Display Time
5.Exit
2
Enter Minutes:
80
1. Add Hours
2. Add Minutes
3. Add Seconds
4. Display Time
5.Exit
3
Enter Seconds:
80
1. Add Hours
2. Add Minutes
3. Add Seconds
4. Display Time
5.Exit
4
Updated Time:
HH: 14 MM: 43 SS: 53
```

10. **Write a menu driven program to perform the following options using separate functions. Accept a number and then, do the following operations on this number:**
    1. **Prime number or not**
    2. **Odd or Even**
    3. **Positive or Negative**
    4. **Exit**

Code:-

```
#include <iostream>
using namespace std;

// 1. Function to check if a number is Prime
void checkPrime(int n) {
    if (n <= 1) {
        cout << n << " is not a prime number." << endl;
        return;
    }
    bool isPrime = true;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            isPrime = false;
            break;
        }
    }
    if (isPrime)
        cout << n << " is a prime number." << endl;
    else
        cout << n << " is not a prime number." << endl;
}

// 2. Function to check Odd or Even
void checkOddEven(int n) {
    if (n % 2 == 0)
        cout << n << " is Even." << endl;
    else
        cout << n << " is Odd." << endl;
}

// 3. Function to check Positive or Negative
void checkPositiveNegative(int n) {
    if (n > 0)
        cout << n << " is Positive." << endl;
    else if (n < 0)
        cout << n << " is Negative." << endl;
```

```
    else
        cout << "The number is Zero." << endl;
}

int main() {
    int num, choice;

    cout << "Enter a number to perform operations on: ";
    cin >> num;

    do {
        cout << "\n--- Menu Driven Program ---" << endl;
        cout << "1. Prime number or not" << endl;
        cout << "2. Odd or Even" << endl;
        cout << "3. Positive or Negative" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice (1-4): ";
        cin >> choice;

        switch (choice) {
            case 1:
                checkPrime(num);
                break;
            case 2:
                checkOddEven(num);
                break;
            case 3:
                checkPositiveNegative(num);
                break;
            case 4:
                cout << "Exiting program..." << endl;
                break;
            default:
                cout << "Invalid choice! Please try again." << endl;
        }
    } while (choice != 4);

    return 0;
}
```

O/P:-

```
Enter a number to perform operations on: 11

--- Menu Driven Program ---
1. Prime number or not
2. Odd or Even
3. Positive or Negative
4. Exit
Enter your choice (1-4): 1
11 is a prime number.

--- Menu Driven Program ---
1. Prime number or not
2. Odd or Even
3. Positive or Negative
4. Exit
Enter your choice (1-4): 2
11 is Odd.

--- Menu Driven Program ---
1. Prime number or not
2. Odd or Even
3. Positive or Negative
4. Exit
Enter your choice (1-4): 3
11 is Positive.

--- Menu Driven Program ---
1. Prime number or not
2. Odd or Even
3. Positive or Negative
4. Exit
Enter your choice (1-4): 4
Exiting program...
```

11. **Write a function reverse() that reverses the character strings. Use it to write a program that reverses its input a line at a time.**

Code:-

```
#include <cstring>
using namespace std;

int getLength(char* str) {
    int len = 0;
    while (str[len] != '\0') {
        len++;
    }
    return len;
}

void reverse(char* str) {
    int start = 0;
    int end = getLength(str) - 1;

    while (start < end) {
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}

int main() {
    int capacity = 10;
    char* buffer = new char[capacity];
    char ch;
    cout << "Enter a string: " << endl;

    while (cin.get(ch)) {
        int i = 0;

        while (ch != '\n' && i < capacity - 1) {
            buffer[i++] = ch;
            if (!cin.get(ch))
                    break;
        }

        buffer[i] = '\0';
```

```
        if (i > 0) {
            reverse(buffer);
            cout << "Reversed: " << buffer << endl;
        }
    }
    delete[] buffer;
    return 0;
}
```

O/P:-

```
Enter a string:
sanket
Reversed: teknas
```

12. **Write a C++ Program to Compare Two given Strings and display the result.**

Code:-

```
#include <iostream>
#include <cstring>
using namespace std;

int compareStrings(char* s1, char* s2) {
    while (*s1 != '\0' || *s2 != '\0') {
        if (*s1 != *s2) {
            return (int)(*s1 - *s2); // Return the difference of ASCII values
        }
        s1++;
        s2++;
    }
    return 0;
}

int main() {
    int size1, size2;
    cout << "Enter max length for String 1: ";
    cin >> size1;
    char* str1 = new char[size1 + 1];
    cout << "Enter max length for String 2: ";
    cin >> size2;
    char* str2 = new char[size2 + 1];
    cout << "Enter String 1: ";
    cin >> str1;
    cout << "Enter String 2: ";
    cin >> str2;

    int result = compareStrings(str1, str2);

    cout << "\n--- Result ---" << endl;
    if (result == 0) {
        cout << "The strings are identical." << endl;
    } else {
        cout << "The strings are different." << endl;
        if (result > 0)
            cout << "String 1 is greater than String 2." << endl;
        else
            cout << "String 2 is greater than String 1." << endl;
    }
    delete[] str1;
```

```
    delete[] str2;
    return 0;
}
```

O/P:-

```
Enter max length for String 1: 10
Enter max length for String 2: 10
Enter String 1: sanket
Enter String 2: shubham

--- Result ---
The strings are different.
String 2 is greater than String 1.
```

13. **Write a program in C++ to determine how many of the characters are vowels and how many are consonants in a given line of text. Also terminate the string when the input character encountered is other than the alphabets (a-z or A-Z) and Blank spaces. Hint:**
    a. When the input string is 'C plus plus FOR BEGINNERS, TEST YOUR C plus plus PROGRAMMING STRENGTHS'. The program will consider the string 'C plus plus FOR BEGINNERS' only because',' is encountered.
    b. When the input string is 'C++ LEARINING IS NOT A PROBLEM'. The program will consider the character 'C' only because the '+' is encountered.

Code:-

```
#include <iostream>
#include <string>
#include <cctype> // for isalpha and tolower
using namespace std;

int main() {
    string input;
    int vowels = 0, consonants = 0;
    cout << "Enter a line of text: ";
    getline(cin, input);
    cout << "Processed String: ";
    for (char c : input) {
        // Terminate if character is NOT an alphabet and NOT a space
        if (!isalpha(c) && c != ' ') {
            break;
        }
        cout << c;
        if (isalpha(c)) {
            char lowC = tolower(c);
            if (lowC == 'a' || lowC == 'e' || lowC == 'i' || lowC == 'o' || lowC == 'u') {
                vowels++;
            } else {
                consonants++;
            }
        }
    }

    cout << "\n\n--- Results ---" << endl;
    cout << "Vowels: " << vowels << endl;
    cout << "Consonants: " << consonants << endl;
    return 0;
}
```

O/P:-

```
Enter a line of text: C plus plus FOR BEGINNERS, TEST YOUR C plus plus PROGRAMMING STRENGTHS
Processed String: C plus plus FOR BEGINNERS

--- Results ---
Vowels: 6
Consonants: 15
```

14. **Create a C++ class for a stock item abstract data type. It should have the attributes of stock levels (an integer) and unit price (a float). Define the methods to return the values of these two attributes and to set them using parameters. Add two more methods to allow stock receipt and issues updating the stocklevel as appropriate. Write menu driven C++ program to solve the problem.**

Code:-

```
#include<iostream>
using namespace std;

class Stock
{
   int stockLevel;
   float unitPrice;

public:

   // set values
   void setValues(int s, float p)
   {
      stockLevel = s;
      unitPrice = p;
   }

   // return stock
   int getStockLevel()
   {
      return stockLevel;
   }

   // return price
   float getUnitPrice()
   {
      return unitPrice;
   }

   // add stock
   void receipt(int qty)
   {
      stockLevel = stockLevel + qty;
      cout << "Stock updated after receipt.\n";
   }

   // remove stock
```

```
    void issue(int qty)
    {
        if(qty <= stockLevel)
        {
            stockLevel = stockLevel - qty;
            cout << "Stock issued successfully.\n";
        }
        else
        {
            cout << "Not enough stock available.\n";
        }
    }
};


int main()
{
    Stock item;

    int choice;
    int stock, qty;
    float price;

    do
    {
        cout << "\n------ MENU ------\n";
        cout << "1. Set Stock Details\n";
        cout << "2. Display Stock Details\n";
        cout << "3. Stock Receipt\n";
        cout << "4. Stock Issue\n";
        cout << "5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch(choice)
        {
            case 1:
                cout << "Enter stock level: ";
                cin >> stock;

                cout << "Enter unit price: ";
                cin >> price;

                item.setValues(stock, price);
```

```
            break;

        case 2:
            cout << "Stock Level = "
                << item.getStockLevel() << endl;

            cout << "Unit Price = "
                << item.getUnitPrice() << endl;
            break;

        case 3:
            cout << "Enter quantity received: ";
            cin >> qty;

            item.receipt(qty);
            break;

        case 4:
            cout << "Enter quantity issued: ";
            cin >> qty;

            item.issue(qty);
            break;

        case 5:
            cout << "Exiting program...\n";
            break;

        default:
            cout << "Invalid choice\n";
    }

  } while(choice != 5);

  return 0;
}
```

O/P:-

```
------ MENU ------
1. Set Stock Details
2. Display Stock Details
3. Stock Receipt
4. Stock Issue
5. Exit
Enter choice: 1
Enter stock level: 200
Enter unit price: 350

------ MENU ------
1. Set Stock Details
2. Display Stock Details
3. Stock Receipt
4. Stock Issue
5. Exit
Enter choice: 2
Stock Level = 200
Unit Price = 350

------ MENU ------
1. Set Stock Details
2. Display Stock Details
3. Stock Receipt
4. Stock Issue
5. Exit
Enter choice: 3
Enter quantity received: 50
Stock updated after receipt.

------ MENU ------
1. Set Stock Details
2. Display Stock Details
3. Stock Receipt
4. Stock Issue
5. Exit
Enter choice: 4
Enter quantity issued: 50
Stock issued successfully.
```

15. Raising a number n to power p is the same as multiplying by itself t times. Write a function called power() that makes a double value for n and an int value for p and returns the result as double value. Use default argument of 2 for p, so that if this argument is omitted, the number will be squared. Write a main() function that gets values from the user to test this function.
Code:-

```
#include <iostream>
using namespace std;
double power(double n, int p = 2) {
 double result = 1.0;
    if (p == 0)
          return 1.0;
    for (int i = 1; i <= p; i++) {
        result *= n;
```

```
    }
    return result;
}

int main() {
    double num;
    int exp;
    char choice;
    cout << "Enter the base number (n): ";
    cin >> num;
    cout << "Do you want to enter an exponent? (y/n): ";
    cin >> choice;
    if (choice == 'y' || choice == 'Y') {
        cout << "Enter the power (p): ";
        cin >> exp;
        cout << num << "^" << exp << " = " << power(num, exp) << endl;
    }
    else {
        cout << num << " squared (default) = " << power(num) << endl;
    }
    return 0;
}
```

O/P:-

```
Enter the base number (n): 1.5
Do you want to enter an exponent? (y/n): y
Enter the power (p): 4
1.5^4 = 5.0625
```